

CaseTrace

项目全景、当前进度 与下一步开发

基于实际仓库内容的导师讲解 · 2026 年 9 月 15 日

数据模型与确定性校验已经建立，第一条本地 BM25 检索流程已经跑通。接下来要把“能返回结果”推进到“能判断结果好不好”。

阅读范围：README、AGENTS、全部项目文档、运行代码、测试、开发样例和主数据 Excel。实际代码目录为 `src/casetrace/`。

01 项目全景图

02 目前实际进度

03 核心代码怎样连接

04 一条真实的数据流

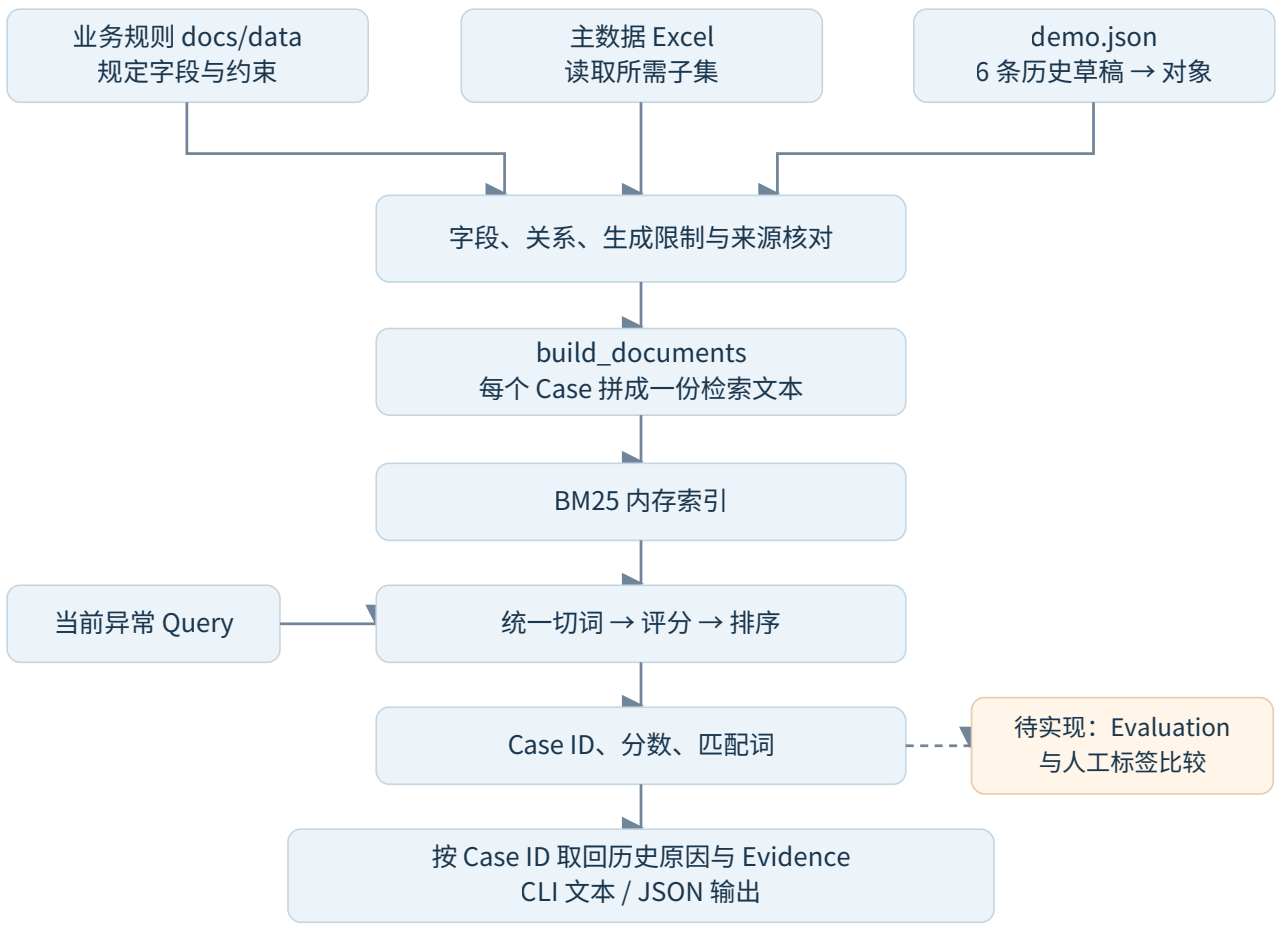
05 当前真正需要掌握什么

06 接下来最合理的 4 个开发步骤

本 PDF 整理自本次对话中的完整讲解。状态与验证结果对应原始仓库审阅时点；原始审阅未修改代码或创建文件。本次仅将讲解导出为 PDF。

1. 项目全景图

当前真正运行的流程



这里有三条需要分开的线：

- **规则线：** 规定什么数据合法。
- **运行线：** 把历史记录变成可查询的内容。
- **评估线：** 判断检索结果是否值得工程师参考，目前尚未接上。

每一层为什么存在

层	输入 → 处理 → 输出	为什么需要、怎样连接
业务规则	项目需求 → 定义字段、关系和允许范围 → 结构文档、CR、GR	给模型与校验器提供依据；Markdown 本身不会在运行时自动执行
主数据	Excel → 读取产品、客户归属、路线、异常知识 → ReferenceData	回答“这个产品、异常是否存在，是否适配”；也提供名称及原因、措施候选
数据模型	JSON 中的字段 → 构造 dataclass、解析日期 → 五类实体列表	让后续代码以一致方式访问记录

层	输入 → 处理 → 输出	为什么需要、怎样连接
Validator	实体列表 + 主数据映射 → 检查字段和跨记录约束 → 错误列表	防止错误数据进入检索；通过检查不等于业务语义已经正确
检索文本构造	Case + 所属 Detail/Evidence + 主数据名称 → 按 Case 汇总文本 → case_id → text	BM25 需要文本，而原始调查内容分散在多类记录中
BM25	历史文本 + 查询 → 切词、评分、排序 → SearchHit 列表	从整个历史语料中找出有词项匹配的案例
结果组装	命中的 Case ID → 查回历史字段和证据 → 可追溯结果	排名只告诉你“看哪个”；原始内容帮助你判断“有什么参考价值”
Evaluation, 待实现	排名 + 人工相关性标签 → 比较 → 指标与错误案例	判断检索质量，为后续改进提供依据

Database 和后续模型应该放在哪里？

Database 是存储基础，不是“Evaluation 之后再处理结果”的一道工序。

现在：

JSON / Excel → Python 内存对象 → 临时 BM25 索引

未来数据库承担历史记录等数据的持久保存，搜索索引从这些记录构建。索引可以重建，不能代替原始数据。

Embedding、Hybrid、Reranker 则扩展检索部分：



这些能力在文档中有规划，目前尚未实现。当前最值得补上的连接是 **排名 → Evaluation**。

2. 目前实际进度

已完成并基本可用

内容	仓库中的实际证据
Python 项目基础	pyproject.toml、uv.lock、CLI 入口和 pytest 配置已存在
五类数据模型	Case、CaseDetail、EvidenceCheckpoint、CaseGroup、Membership 已实现
确定性校验	字段、ID、引用、父子数量、批号一致性、路线适配，以及部分 GR 已实现
演示所需主数据读取	reference.py 能读取必要 Sheet，检查重复 ID、归属覆盖和路线引用
本地 BM25	已实现切词、索引构建、查询、排序和来源 ID 保留
最小 CLI 流程	能读取真实仓库样例，校验后检索，并展示历史原因和证据

直接运行依赖只有 `openpyxl` 和 `rank-bm25`，当前流程不需要调用 LLM。

已有雏形／仍需完善

内容	准确状态
开发数据	6 条 Case、6 条 Detail、6 条 Evidence、3 条 Query；全部仍标记待人工审阅
来源核对	已保留原因／措施候选来源，并做机械核对；尚不能证明改写后的语义完全一致
主数据导入	演示子集可用；完整 BOM、设备、工序关系导入校验尚未实现
Current Incident	已有查询文本和 known_at 元数据；没有独立的结构化 Incident 模型或输入抽取流程
来源追溯	可追到 Case ID、Evidence ID；完整运行版本、抽取来源关系等尚未建立
BM25 基线工作	实现能运行，但还没有人工 Ground Truth 支撑的质量结论

Excel 实际包含 **12 张 Sheet、498 条有效记录**，其中有 18 个产品、37 种 Failure Mode、77 条固定 BOM 关系。

这些是环境和候选知识，不是 498 条历史 Case。当前历史 Case 只有 6 条。

尚未实现

- 检索 Ground Truth、指标计算、实验结果和正式 Error Analysis；`evaluation/__init__.py` 为空。
- Locked Test 数据及实际划分流程。
- 自动批量 Case 生成器与完整语义验收。

- PostgreSQL Schema、数据库访问层、Elasticsearch/OpenSearch 接入。
- Embedding、Hybrid、Metadata filtering/boosting、Reranker。
- A/C/D 的 AI Extraction、Relevance Type 和 Rationale 生成。
- Web Backend、Frontend、Docker、CI 和完整运行追踪。

怎样读文档里的“完成”？

有几处容易误解：

1. **Stage 5 的“已完成”表示技术设计完成。** 不表示数据库、前后端、Hybrid 已开发完成。
2. **Stage 6、7 是方案冻结。** 不能把里面的最终验收清单当成现有功能。
3. **9月14日的 review 文件是历史快照。** 其中“没有主数据读取/检索命令”的描述，已被现在的实现推进。
4. Stage 4 中的 `Scenario-first` 可以理解为“先确定事实和关系，再组织文本”；具体数据结构以 `docs/data` 的 **无 Scenario 实体** 决定为准。

当前最贴近实际迭代的是 Stage 6 § 6.1.2：少量草稿 → 本地 BM25 → 人工标注 → 评估。

来源：docs/project/stage6.md:114

原始审阅中的实际验证

- 收集到 **116 个正式测试**。
- 执行了不需要创建文件的部分：**104 passed, 169 subtests passed**。
- 另外 12 个使用临时 JSON/Excel 的测试只阅读、未执行，以遵守原始审阅“不创建文件”的要求。
- 实际运行了仓库样例的 CLI 和三条内置查询。

这些结果证明所测软件行为能运行，尚不能证明检索质量达标。

3. 核心代码怎样连接

① dataset_model.py：描述调查记录的结构

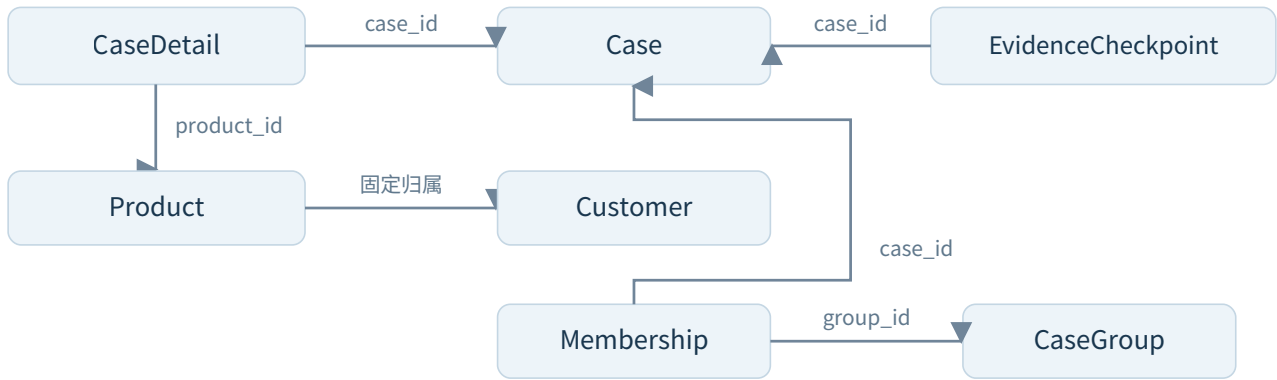
源码：src/casetrace/data/dataset_model.py

模块 → 字段值 → 构造记录对象、通过 ID 表达关系 → 五类 dataclass → 为校验和检索提供统一的数据表示。

五类对象分别回答：

对象	回答的问题
Case	这次调查发生了什么，最后确认了什么原因、采取什么措施？
CaseDetail	涉及哪个产品、哪个批号、哪次发现事件、多少处置数量？
EvidenceCheckpoint	做了什么调查，结果是什么，与历史原因是否相关？
CaseGroup	哪些 Case 被显式组织为一组？
Membership	某个 Case 为什么加入这个 Group？

关系可以这样读：



最重要的理解是：

一份完整调查由 Case + 对应的 Details + 对应的 Evidences 组成。

Case 自身没有保存全部子记录，也没有重复保存它们的 ID 列表。要找到某 Case 的 Evidence，就按 Evidence 的 case_id 筛选。

此外，当前五份列表共同构成一次校验的 Dataset；没有额外的 Dataset 类。

② validators.py：把规则变成可执行检查

源码：src/casetrace/data/validators.py

模块 → 实体列表及必要映射 → 分阶段检查确定性约束 → list[str] 错误列表 → 为下游提供基本可信的数据。

三个入口的分工：

入口	主要检查
validate_records()	单条记录的必填、类型、枚举、日期顺序等
validate_relations()	先检查字段，再检查 ID、引用、子记录数量、批号一致性和主数据关联
validate_generation()	字段守门后，检查已实现的合成数据数量限制和批号复用规则

演示中的顺序是：

```
validate_relations()
    ↓ 通过
validate_generation()
    ↓ 通过
来源核对
```

两个特别值得掌握的设计：

第一，检查范围必须足够。

“数量是不是正整数”看一条 Detail 就够了；“同一批号的产品是否一致”必须比较整份 Dataset。

第二，CR 与 GR 是两种问题。

例如 affected_qty=5001：

- 符合 CR 的“正整数”要求；
- 不符合 GR 的“生成数量不超过 5000”要求。

validate_generation() 的名字也不表示它会生成数据，它只检查生成限制。

返回 [] 表示“这个入口负责的检查没发现错误”。它无法证明“证据真的支持原因”或“两个案例确实复发”。

③ reference.py：连接 Excel 与运行代码

源码：src/casetrace/data/reference.py

模块 → Excel 路径 → 读取必要列、检查基本引用、构造映射 → ReferenceData → 同时支持 Validator 和检索文本构造。

它有两个下游：

```
ReferenceData
├─ validator_maps() → 产品客户、产品路线、异常适用路线
└─ 名称与候选文本 → 文本构造、来源核对
```

例如：

```
PROD_005 → LF_WB
00018 → molding void
```

Validator 用它判断异常是否适配产品路线；`build_documents()` 用它把 ID 对应的名称加入检索文本。

所以主数据读取的作用不只是“打开 Excel”，还包括把存储格式转换成运行模块需要的接口。

④ demo.py：当前流程的连接中心

源码：src/casetrace/demo.py

模块 → 数据路径、主数据路径、查询和 K → 加载、校验、构建文本、调用检索、组装结果 → 结果字典 → 串起当前完整演示。

三个关键函数：

- `load_demo()`：JSON → dataclass；日期字符串 → date。
- `build_documents()`：分散的调查记录 → 每个 Case 一份文本。
- `run_demo()`：控制执行顺序，并把搜索结果还原成可阅读的历史内容。

其中 `build_documents()` 是数据工程与检索之间的关键接口。

它决定 BM25 实际能看见什么：

- 历史异常描述、结案原因、措施；
- 产品、路线、批号、日期等内容；
- Failure Mode 名称；
- Evidence 名称与结果。

查询、来源审查元数据、候选库全文和 Group 内容没有被拼进索引。内部 Case/Detail/Evidence ID 用于追溯，也没有作为检索词。

产品与日期虽然进入了文本，当前只是普通词项，并没有变成结构化过滤或时间推理。

⑤ retrieval/bm25.py：只负责文本检索

源码：src/casetrace/retrieval/bm25.py

模块 → case_id → text 和 Query → 统一词项、BM25 评分、排序 → list[SearchHit] → 提供可复用的检索核心。

它不知道 JSON、Excel、Validator，也不读取 Ground Truth。

具体行为：

- 中英文都先规范化，英文转小写。
- 中文采用相邻双字切分。
- 通过 `BM25Okapi` 评分。
- 没有任何词项交集的 Case 不返回。

- 分数相同时按 Case ID 排序。
- 返回最多 K 条，不保证凑满 K 条。

SearchHit 只包含：

```
case_id + score + matched_terms
```

匹配词解释的是“哪些词重合”，还不能解释“为什么有工程参考价值”。

⑥ init.py 与 pyproject.toml：把命令接到函数

CLI 源码：src/casetrace/__init__.py:4

入口配置：pyproject.toml:15

模块 → 命令行参数 → 解析参数、调用 run_demo()、格式化结果 → 终端文本 / JSON → 提供使用入口。

关键配置只有：

```
[project.scripts]
casetrace = "casetrace:main"
```

它把命令 casetrace 连接到 Python 包里的 main()。

检索逻辑保留在独立模块中，因此未来评估程序可以直接调用相同检索代码。

测试在这里扮演什么角色？

测试是运行流程旁边的检查工具，不是每次查询必经的步骤。

当前测试主要保护：

- 规则边界是否正确；
- 错误输入是否被拒绝；
- 检索是否保留来源；
- 查询和审查信息是否意外进入索引。

tests/data/test_dataset_model.py 只有占位注释，但模型已经在大量 Validator 测试中被使用。文件名或文件存在本身都不能代表功能完成。

4. 一条真实的数据流

使用这个最简单的查询：

```
uv run casetrace demo --query "塑封空洞 molding void" --top-k 3
```

原始审阅通过现有虚拟环境直接执行了同一 CLI 入口，避免环境同步或缓存写入。

第一步：命令进入 main()

uv 负责在项目环境中运行入口；业务处理从 `main()` 开始。

`main()` 得到：

```
query = "塑封空洞 molding void"
top_k = 3
data_path = data/dev/demo.json
reference_path = 主数据 Excel
```

然后调用 `run_demo()`。

第二步：先加载整份历史语料

程序不会直接寻找 C005。它先读入全部 6 条 Case 及子记录。

其中 C005 的记录包括：

来源	内容
Case C005	声学扫描发现塑封空洞；历史结案原因为 mold vent/vacuum不足
Detail D005	产品 PROD_005，异常 ID 00018，处置范围 500 ea
Evidence E005	SAM 确认边缘空洞；排气通道堵塞，清理后对照试模空洞减少

JSON 中的日期在 `load_demo()` 内转换为 `date`，其他字段用于构造 `dataclass`。

第三步：加载主数据并校验

主数据补充：

```
PROD_005 → SYN_MCU_A → LF_WB
00018 → molding void → 适用于 LF_WB
```

随后检查整份 Dataset：

- 字段及日期是否合法；
- D005、E005 是否引用存在的 C005；
- 产品、异常、路线是否有效；
- 数量及其他已实现 GR 是否满足；
- 所选原因和措施是否仍在 Excel 的候选中；
- 标记 `confirmed` 的草稿是否至少有一条 `related Evidence`。

这里需要整份 Dataset，还有一个具体原因：C001 与 C002 复用了 `DEV_PL_001`，批号固定属性和复用数量都必须跨记录检查。

第四步：把 C005 拼成检索文本

`build_documents()` 汇总：

```
C005 的异常、原因、措施
+ D005 的产品、批号、日期等
+ 产品名称与 molding void 名称
+ E005 的检查名称与结果
```

输出的形状是：

```
{
  "C001": "...",
  ...
  "C005": "...塑封空洞 molding void...",
  ...
}
```

这是一份**从原始记录派生出来的搜索表示**。原始对象仍然保留。

第五步：构建 BM25 并检索

查询变成：

```
["塑封", "封空", "空洞", "molding", "void"]
```

历史文档也采用相同切词规则。

BM25 根据词项在文档和语料中的分布评分，随后排序。当前每次运行 demo 都会重新读取数据、构建内存索引，没有持久化索引服务。

第六步：按命中的 ID 查回原始内容

原始审阅中的实际结果：

```
C005
BM25 = 9.121
匹配词 = molding, void, 塑封, 封空, 空洞
历史结案原因 = mold vent/vacuum不足
来源 = E005
```

虽然 `top_k=3`，这里只返回一条，因为只有 C005 与这条查询存在词项交集。

`run_demo()` 根据 C005 查回 `root_cause` 和所属 Evidence，CLI 再打印出来。

这里没有发生 LLM 原因抽取或原因预测。显示的是 C005 已保存的历史结案原因。当前程序也没有建立 Cause→Checkpoint 抽取关系，而是取回命中 Case 下的 Evidence。

两个实际边界

查询的时间边界尚靠输入约束维护。 `known_at` 已保存在三条示例查询中，但运行代码不使用它过滤历史语料，也不自动检查用户有没有输入未来才知道的原因。

BM25 尚不能理解否定。 实际运行 Q001 时，前三名是 C001、C003、C004。C004 的证据写着“未观察到 pad 界面剥离或污染”，其中“剥离”等词仍然可能参与匹配。C004 是否应被判为不相关，需要人工判断，不能仅凭排名决定标签。

5. 当前真正需要掌握什么

你已有的 Python 和检索基础足够继续。现在重点是理解接口、数据角色和判断边界。

Python：掌握到能跟踪对象流动

知识	需要达到的程度
list、dict、set	能解释为什么记录用列表、按 ID 查询用字典、唯一性及交集用集合
函数参数与返回值	能读懂 <code>model(values)</code> 、 <code>validate_relations(records, ...)</code> ，知道字典中的键如何对应参数
JSON 与 Python 对象转换	知道日期字符串要在加载层解析，dataclass 构造成功不代表合法
异常与错误列表	能解释 Validator 为什么返回多条错误，而 <code>run_demo()</code> 为什么遇到失败就停止
<code>sorted()</code> 与排序键	能读懂分数降序、Case ID 同分排序；无需研究复杂排序算法

项目结构：掌握到知道改动应该放哪里

知识	需要达到的程度
规则与实现的关系	规则问题查 docs/data；不能因为测试通过就认为业务规则一定正确
数据文件与运行模块	data/ 保存内容，src/casetrace/data/ 保存处理内容的代码
入口与核心逻辑	CLI 负责接收和展示；检索核心应能被 demo 与评估共同调用
实体与派生表示	能区分 Case 原始记录、拼接文本、搜索索引、搜索结果
测试范围	知道单元测试、流程测试各证明什么；不把它们当成 AI 质量评估

检索与评估：掌握到能判断一次实验有没有意义

知识	需要达到的程度
Corpus、Query、qrels	能区分历史案例库、当前问题、人工相关性判断表
文本构造与切词	能检查“想匹配的词是否进入索引，查询和文档是否切成相同词项”
Recall@K、Precision@K	能在一个人工标注的小例子上手算，并解释漏检与误检
分数与相关性	知道 BM25 分数用于排序，不是概率，也不是人工标签
数据泄漏	能检查查询是否偷用了未来原因，索引是否混入标签或审查线索

知识	需要达到的程度
Development／Locked Test	知道开发集可以反复调试；Locked Test 留作独立检验

特别注意三个“相关”的含义：

Evidence.relevance

→ 这条历史证据与该 Case 的原因是否相关

Query-Case Ground Truth

→ 这个历史 Case 对当前查询是否值得参考

CaseGroup

→ 人工显式建立的 Case 组织关系

三者不能互相自动替代。

暂时可以不会

- BM25 公式推导、Transformer 内部数学、模型训练。
- 完整 PostgreSQL 并发控制、搜索引擎运维。
- FastAPI、React、复杂 async、Docker 部署细节。
- 自动合成数据的完整生成平台。

当前先达到一个目标就够了：**能指出一次错误结果来自数据、文本构造、切词、排序还是标注。**

6. 接下来最合理的 4 个开发步骤

1. 人工审阅现有 6 条草稿和 3 条查询

做什么： 核对描述、Detail、证据、原因和措施是否一致，确认查询只包含当时已知事实。

为什么现在做： 当前数据明确仍是草稿；不可信的样例会让后面的指标失去意义。

完成标准： 每条 Case 和 Query 都有明确的保留、修正或待确认结论；保留项可追溯到来源。

2. 建立第一份 Development qrels

做什么： 对保留的 Query-Case 组合逐对标注 Relevant / Not Relevant / Ambiguous，记录类型、理由和依据。若全部保留，共 18 对。

为什么现在做： 已有排名，但还没有判断排名是否正确的标准。

完成标准： 每个组合都有经过人工确认的判断；标签放在检索输入之外，不根据 BM25 分数反推标签。

3. 接上最小 BM25 Evaluation

做什么： 复用现有 `build_documents()` 和 `BM25Retriever`，批量运行查询，与 qrels 比较；先实现 Recall@K、Precision@K。

为什么现在做： 把“感觉结果不错”变成可以复查、可以比较的结果。

完成标准： 能重复输出每条查询及整体指标；明确 K、Ambiguous、无正例及返回不足 K 条时的计算口径；用手算小例验证计算正确。

4. 完成第一轮错误分析，选择一个具体改进

做什么： 检查漏掉了哪些案例、哪些表面相似案例排得过高，并定位原因；证据明确后只改变一个因素。

为什么现在做： 需要先知道 BM25 的实际弱点，才能判断下一项改进是否有价值。

完成标准： 保存首版基线结果，解释代表性错误；如做改进，用同一份开发数据比较前后变化。

下一次最合适的交付物

一份经过人工审阅、带相关性标签、能运行 BM25 评估的小开发集。